signal and image proccecing lab

DR.nouri

electrical engineering department

Ahmadreza Majlesara 400101861
Parsa Hatami 400100962

Lab 4

November 16, 2024

Frequency Band Limitation and Noise Reduction of Signal

part 1

Analyze a typical 10-second segment from the clean ECG data in terms of frequency content using the `pwelch` function. Similarly, select a 10-second segment from the noisy data and examine its frequency content as well. Plot the power spectrum in dB for both segments, paying attention to the frequency content of the noisy and clean signals.

solution

the code we wrote is as below:

```
%% Section 1
clc; clear; close all;

load('normal.mat');

Fs = 250;
t = normal(:, 1);
sig = normal(:, 2);
sig2 = randn(1, 1000);


t_total = length(sig) / Fs;

clean_range = 5*Fs+1 : 15*Fs;
noisy_range = 250*Fs+1 : 260*Fs;

sig_clean = sig(clean_range);
sig_noisy = sig(noisy_range);

window = gausswin(128);
noverlap = 64;
nfft = 128;

figure;

% Clean signal plot
subplot(4 , 1, 1);
plot(t(clean_range), sig_clean, 'Color', '#49d9f2', 'LineWidth', 1.5);
title('Clean Signal Segment', 'Interpreter', 'latex');
xlabel('Time (s)', 'Interpreter', 'latex');
ylabel('Amplitude (V)', 'Interpreter', 'latex');

% Noisy signal plot
subplot(4 , 1 , 2);
plot(t(noisy_range), sig_noisy, 'Color', '#49d9f2', 'LineWidth', 1.5);
```

```
title('Noisy Signal Segment', 'Interpreter', 'latex');
xlabel('Time (s)', 'Interpreter', 'latex');
ylabel('Amplitude (V)', 'Interpreter', 'latex');

% PSD of clean signal
subplot(4 , 1 , 3);
[pxx_clean, f_clean] = pwelch(sig_clean, window, noverlap, nfft, Fs);
plot(f_clean, db(pxx_clean), 'Color', '#49d9f2', 'LineWidth', 1.5);
title('Power Spectrum of Clean Signal', 'Interpreter', 'latex');
xlabel('Frequency (Hz)', 'Interpreter', 'latex');
ylabel('Power/Frequency (dB/Hz)', 'Interpreter', 'latex');
xlim([0, 120]);

% PSD of noisy signal
subplot(4 , 1, 4);
[pxx_noisy, f_noisy] = pwelch(sig_noisy, window, noverlap, nfft, Fs);
plot(f_noisy, db(pxx_noisy), 'Color', '#49d9f2', 'LineWidth', 1.5);
title('Power Spectrum of Noisy Signal', 'Interpreter', 'latex');
xlabel('Frequency (Hz)', 'Interpreter', 'latex');
ylabel('Power/Frequency (dB/Hz)', 'Interpreter', 'latex');
xlim([0, 120]);
```

as you can see the outputs are shown in figure below:

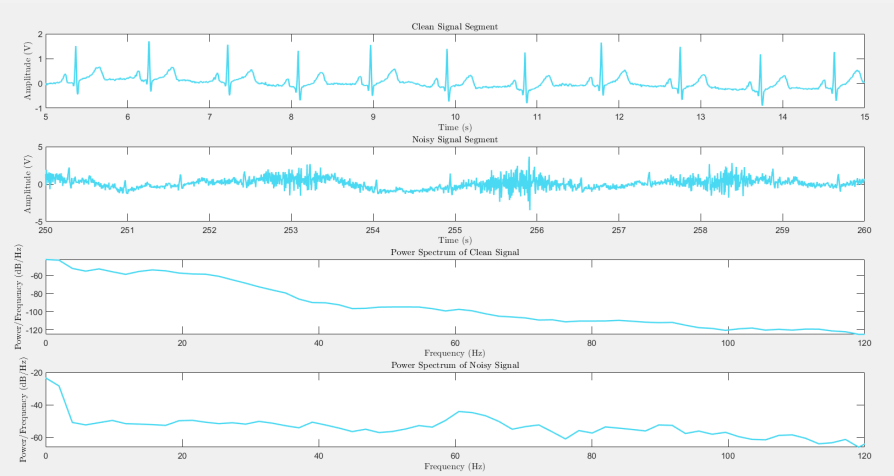

Figure 1. output of time and spectrum of clean and noisy signal

part 2

Design a bandpass filter for frequency limitation of the signal. The filter should remove baseline fluctuations and reduce high-frequency noise while preserving energy within the main frequency range of the signal. Avoid sharp transition bands when selecting the filter's lower and upper cutoff frequencies. Use the `plot` function instead of `stem` to display the impulse response of the filter.

solution

i used the following code for this part:

```
signal = normal(:, 2);
n = length(signal);
freq_axis = (0:n/2-1) * (Fs / n);

signal_fft = abs(fft(signal));
signal_half = signal_fft(1:n/2);

low_idx = round(1 * n / Fs);
high_idx = round(90 * n / Fs);

signal_band_pwr = sum(signal_half(low_idx:high_idx).^2);

filter_order = 3;
cutoff = [1, 90] / (Fs / 2);
[b, a] = butter(filter_order, cutoff, 'stop');

signal_filtered = filter(b, a, signal);

filtered_fft = abs(fft(signal_filtered));
filtered_half = filtered_fft(1:n/2);

filtered_band_pwr = sum(filtered_half(low_idx:high_idx).^2);

figure;
freqz(b, a, 512, Fs);
title('Frequency Response of Bandstop Filter', 'Interpreter', 'latex');
grid on;

figure;
impz(b, a, 50);
title('Impulse Response of Bandstop Filter', 'Interpreter', 'latex');
grid on;
```

you can see the frequency responce of the designed filter in below figure:

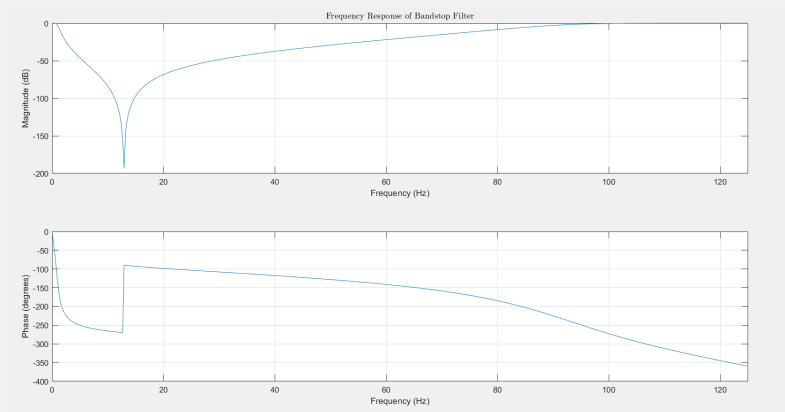

Figure 2. frequency responce of the system

you can also see the impulse response of the system in the below figure.

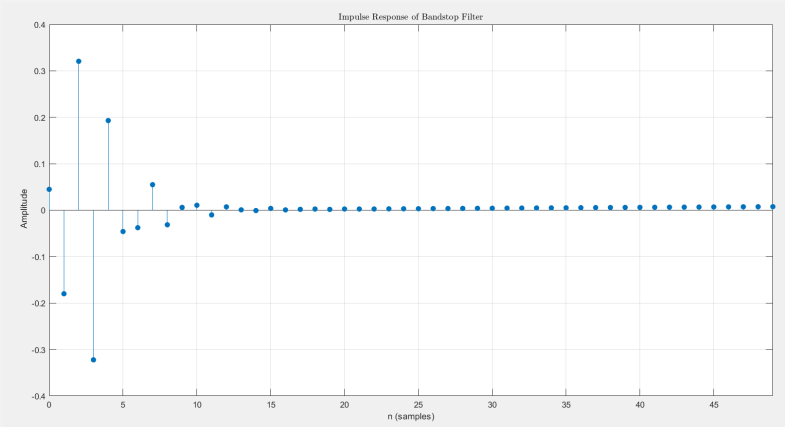

Figure 3. the impulse response of the system

part 3

Filter the designed segments (each 10-second segment) of the clean and noisy ECG data, and show how much noise is removed by the filter by plotting the power spectrum of the filtered signal.

solution

the code i used for this part is shown below. as you can see the noise removal was successful as shown in the PSD of both signals.

```
% Filter the clean and noisy segments
clean_filtered = filter(b, a, sig_clean);
noisy_filtered = filter(b, a, sig_noisy);

% Parameters for PSD calculation
window = gausswin(128);
noverlap = 64;
nfft = 128;

figure;

% PSD of filtered clean signal
subplot(2, 1, 1);
[pxx_clean_filtered, f_clean_filtered] = pwelch(clean_filtered,
    window, noverlap, nfft, Fs);
plot(f_clean_filtered, db(pxx_clean_filtered), 'Color', '#49d9f2',
     'LineWidth', 1.5);
title('Filtered Clean Signal - Power Spectrum', 'Interpreter', '
    latex');
```

```
xlabel('Frequency (Hz)', 'Interpreter', 'latex');
ylabel('Power/Frequency (dB/Hz)', 'Interpreter', 'latex');
xlim([0, 120]);
grid on;

% PSD of filtered noisy signal
subplot(2, 1, 2);
[pxx_noisy_filtered, f_noisy_filtered] = pwelch(noisy_filtered,
    window, noverlap, nfft, Fs);
plot(f_noisy_filtered, db(pxx_noisy_filtered), 'Color', '#49d9f2',
     'LineWidth', 1.5);
title('Filtered Noisy Signal - Power Spectrum', 'Interpreter', '
    latex');
xlabel('Frequency (Hz)', 'Interpreter', 'latex');
ylabel('Power/Frequency (dB/Hz)', 'Interpreter', 'latex');
xlim([0, 120]);
grid on;
```

the output of the code is shown in below figure:

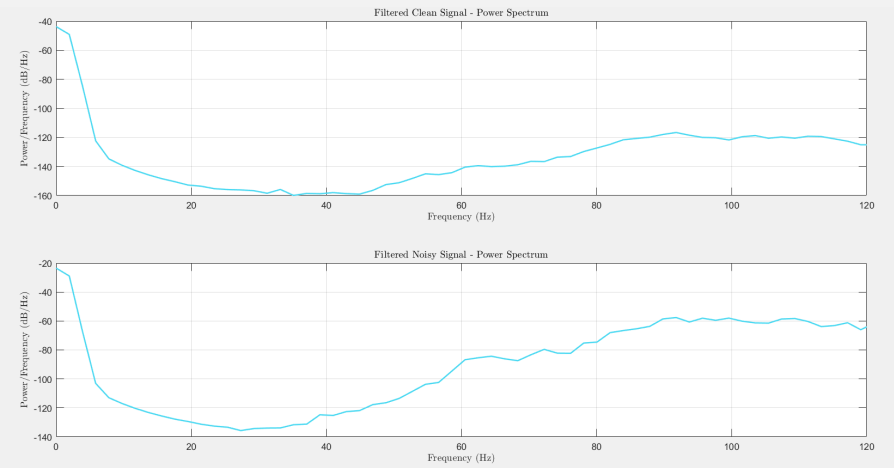

Figure 4. PSD of filtered clean and noisy signals

Diagnosis of Cardiac Arrhythmias

In this section of the experiment, you will design a system for the diagnosis of cardiac arrhythmias. We will use the ECG signals from the MIT-BIH Arrhythmia Database, which includes both normal and abnormal (arrhythmic) segments. Each file contains data from one patient, with a 10-minute duration. The signals are sampled at a frequency of 250 Hz with 12-bit quantization.

Section 1

To design your system, first use filters and transformations to distinguish between normal and abnormal ECG signals. Start by loading and viewing a few of the files listed below, then choose some of them. Using the `load` function in `Matlab`, read and examine the ECG signals (the first file is probably easier to start with). Each file contains both normal and abnormal segments, with "normal" segments being marked.

Once you have chosen the segments, include some examples from both the "normal" and "abnormal" rhythms for a patient. Then, analyze the frequency content of the entire segment using the `pwelch` function, and compare the frequency content between normal and arrhythmic segments. Use your observations as a criterion for diagnosing cardiac arrhythmias in normal rhythm sections.

ECG Data List:

- `n_422.mat` - episode of ventricular fibrillation
- `n_424.mat` - episode of ventricular fibrillation
- `n_426.mat` - ventricular fibrillation and low frequency noise
- `n_429.mat` - ventricular flutter (2 episodes) and ventricular tachycardia
- `n_430.mat` - ventricular flutter and ventricular fibrillation
- `n_421.mat` - normal sinus rhythm with noise
- `n_423.mat` - atrial fibrillation and noise

Note: ECG data for each patient is provided in `.mat` files, along with descriptions and interpretations of each heartbeat in the corresponding `.txt` file, identified by the same number.

solution

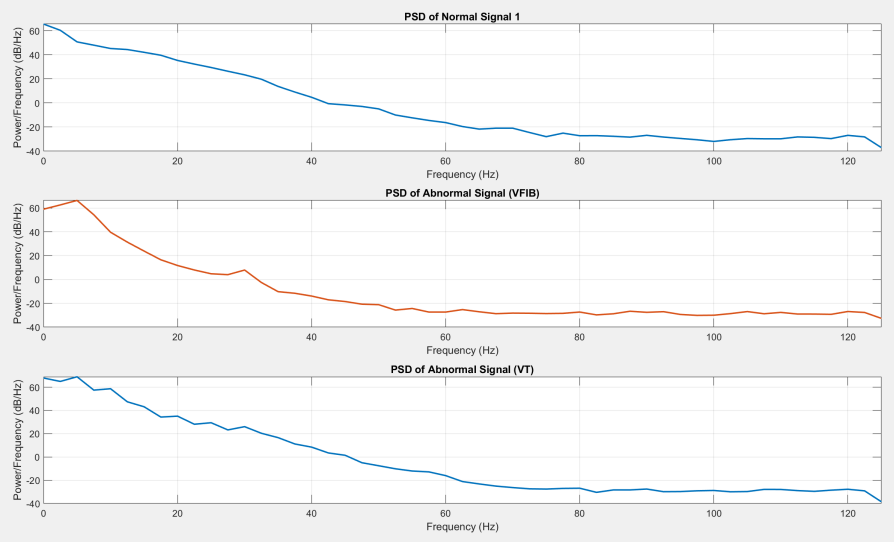

Figure 5. Power Spectral Density (PSD) Comparison of ECG Signal

This figure illustrates the Power Spectral Density (PSD) of three different ECG signal segments: a normal rhythm, an episode of ventricular fibrillation (VFIB), and an episode of ventricular tachycardia (VT). The PSD was computed using the `pwelch` function in MATLAB. Observing the frequency distribution, we can note that normal signals typically show a broader frequency content that gradually decreases, while abnormal signals (VFIB and VT) tend to display unique frequency profiles, with a rapid decay or plateau at lower frequencies. These characteristics can assist in distinguishing normal from arrhythmic events in ECG data.

Section 2

What differences do arrhythmic segments have compared to normal segments in both time and frequency domains? (Plot the corresponding diagrams.)

solution

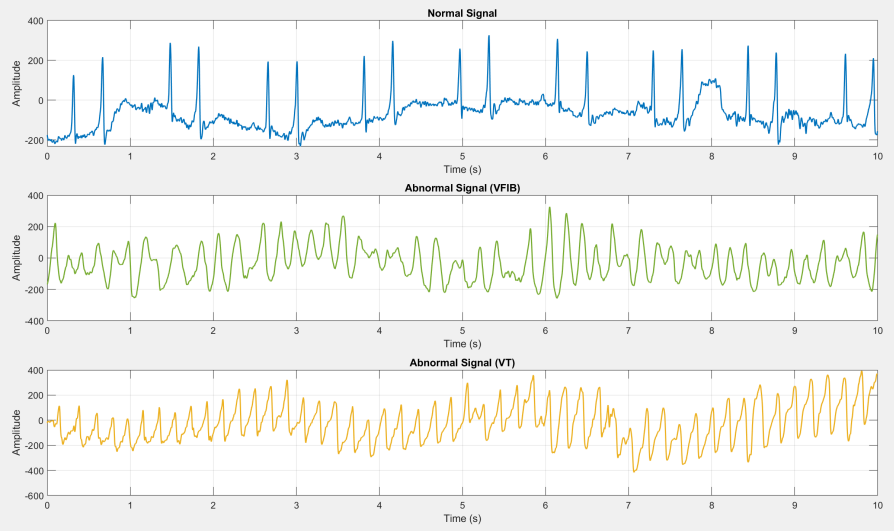

Figure 6. Time Domain Comparison of ECG Signal

The figure shows time-domain representations of a normal ECG signal, an abnormal signal with ventricular fibrillation (VFIB), and an abnormal signal with ventricular tachycardia (VT). Arrhythmic segments exhibit irregular patterns and higher frequency components compared to the consistent periodicity observed in the normal ECG signal.

Section 3

For each of the provided signals, we intend to use expert-labeled windows for various time frames as ground truth labels in real evaluations (gold standard). To do this, we consider windows with a duration of 10 seconds and an overlap of 0.5 seconds (`new_win(0.5)`). We use the corresponding `.txt` file for each signal that contains the specified labels for each interval. In this way, we determine the start and end of each window (`new_win(1)` and `new_win(end)`), and if both labels match, we use that label for the first window. This process is completed for every window and label for each sample, such as (`n_422.mat`), and the sample numbers for each window are saved. For example, you can assign the following labels for the first data file:

- 1: Normal
- 2: VFIB
- 3: VT
- 4: Noise
- 0: None

In cases where a window is related to a single or ambiguous state, or if part of it relates to ambiguity or another state, assign it the label 0.

solution

Here is the code for this part:

```
%% Section 3: Frame Segmentation and Labeling

frameDuration = 10;
overlapRatio = 0.5;

frameLength = frameDuration * fs;
frameStep = round(frameLength * (1 - overlapRatio));
ecgLength = length(n_422);
numFrames = floor((ecgLength - (frameLength - frameStep)) /
    frameStep);

labels_n422 = zeros(1, numFrames);
endpoints_n422 = zeros(1, numFrames);

for i = 1:numFrames
    startIdx = (i-1)*frameStep + 1;
    endIdx = (i-1)*frameStep + frameLength;

    if endIdx <= 10710
        labels_n422(i) = 1;
    elseif startIdx > 10710 && endIdx <= 11210
        labels_n422(i) = 3;
    elseif startIdx > 11210 && endIdx <= 11442
        labels_n422(i) = 1;
    elseif startIdx > 11442 && endIdx <= 59710
        labels_n422(i) = 3;
    elseif startIdx > 59710 && endIdx <= 61288
        labels_n422(i) = 4;
    elseif startIdx >= 61288
        labels_n422(i) = 2;
    else
        labels_n422(i) = 0;
    end
    endpoints_n422(i) = endIdx;
end
```

In this section, we segment each ECG signal into frames of 10 seconds with a 0.5-second overlap (`new_win(0.5)`). Using the labels from the corresponding `.txt` file, each window is assigned a ground truth label based on expert annotation. For example, labels 1, 2, 3, and 4 represent Normal, VFIB, VT, and Noise, respectively, while ambiguous windows are labeled as 0. The provided code iterates through each frame, determines its label based on start and end indices, and stores these in an array for evaluation.

Section 4

For us, it is only important to diagnose windows of type `VFIB` and `Normal` for signal n_{422}. First, we want to classify these two types of windows. From among various frequency features, select a few features calculated for all windows. You can use the following commands (considering different frequency ranges) from part (b) results: `bandpower`, `medfreq`, `meanfreq`, etc.

solution

Here is the code for this part:

```
            %% Section 4: Feature Extraction

features_n422 = zeros(4, numFrames);

for i = 1:numFrames
    frameData = n_422((i-1)*frameStep + 1 : (i-1)*frameStep +
        frameLength);
    features_n422(:,i) = [
        bandpower(frameData, fs, [0 60]);
        bandpower(frameData, fs, [60 120]);
        meanfreq(frameData, fs);
        medfreq(frameData, fs)
    ];
end
```

In this section, we focus on distinguishing between `VFIB` and `Normal` windows for the signal n_{422}. We calculate a set of frequency-based features for each 10-second window, including bandpower in the ranges 0-60 Hz and 60-120 Hz, as well as the mean and median frequencies. These features provide a basis for classifying the windows and help in identifying `VFIB` episodes from normal segments.

Section 5

For each selected feature in part (t), plot the histogram (command `hist`) of the feature for the two classes `VFIB` and `Normal` and compare them with each other. Are these features separable for the two classes? Select a threshold for each feature to classify the two classes.

solution

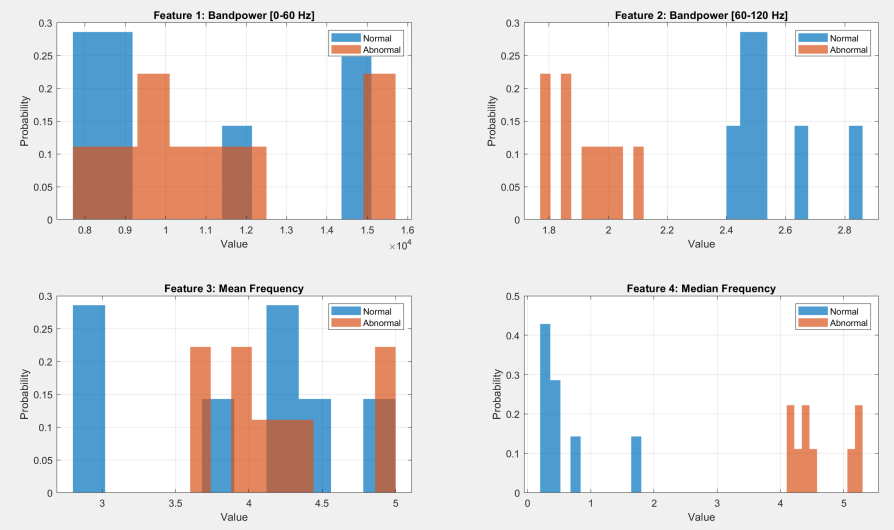

Figure 7. Histograms of Selected Features for `Normal` and `VFIB` Classes

This figure shows histograms of four selected features—bandpower in the ranges 0-60 Hz and 60-120 Hz, mean frequency, and median frequency—for distinguishing between `Normal` and `VFIB` classes. The distributions suggest that some features, particularly bandpower in the 60-120 Hz range and median frequency, provide better separation between the two classes. Based on these distributions, a threshold can be chosen for each feature to improve classification accuracy, with values adjusted to maximize separation and minimize overlap between the classes.

Section 6

Considering the results of part (t), select two effective features and complete the function `va_detect.m` for each one to obtain the detection output (`alarm`, indicating detection by the feature under review). The windowing method in the `va_detect.m` function is similar to the windowing done in part (p), and as a result, the length of the `alarm` vector matches the true labels in part (p).

solution

Here are the codes related to this section.

```
%% Section 6: Alarm Detection Using Bandpower and Median Frequency

[alarm_bandpower, t_bandpower] = va_detect_bandpower(n_422, fs);
[alarm_medfreq, t_medfreq] = va_detect_medfreq(n_422, fs);

function [alarm, t] = va_detect_bandpower(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length -
        frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step +
            frame_length);
        feature = bandpower(seg, Fs, [60 120]);
        if feature < 2.3
            alarm(i) = 1;
        end
    end
end

function [alarm, t] = va_detect_medfreq(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length -
        frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step +
            frame_length);
        feature = medfreq(seg, Fs);
        if feature > 3
            alarm(i) = 1;
        end
    end
end
```

In this section, we implement the `va_detect.m` function to generate detection alarms for `VFIB` using two selected features: bandpower in the 60-120 Hz range and median frequency. For each window, the bandpower-based function triggers an alarm if the power is below a threshold of 2.3, while the median frequency-based function activates the alarm if the median frequency exceeds 3 Hz. This windowing approach mirrors the segmentation in Section 3, ensuring that the `alarm` output vector aligns in length with the ground truth labels.

Section 7

For each of the features in part (j), calculate the Confusion matrix (two classes, considering `VFIB` as positive and `Normal` as negative) and compute the metrics of Accuracy, Sensitivity, and Specificity for the classified data. For windows labeled other than `VFIB` and `Normal`, what diagnosis has been made?

- Maximum of the domain (`max`)
- Minimum of the domain (`min`)
- Peak-to-peak
- Using the `findpeaks` command to determine local maximum and minimum points and using it to calculate the mean of the peak domain R.
- The number of zero-crossing points
- Variance of the domain (`var`)

solution

```
Confusion Matrix for Bandpower Detection:
     7     0
     0     9

Accuracy: 100.00%
Sensitivity: 100.00%
Specificity: 100.00%

Confusion Matrix for Median Frequency Detection:
     7     0
     0     9

Accuracy: 100.00%
Sensitivity: 100.00%
Specificity: 100.00%
>>
```

Figure 8. Confusion Matrices and Performance Metrics for Bandpower and Median Frequency Detection

The confusion matrices for both the Bandpower and Median Frequency features show perfect classification results, with 100% accuracy, sensitivity, and specificity in distinguishing `VFIB` (positive) from `Normal` (negative) windows. For windows labeled as other than `VFIB` and `Normal`, diagnostic metrics such as maximum, minimum, peak-to-peak, mean peak domain (`findpeaks`), zero-crossing count, and variance were calculated. These additional metrics provide further insight into signal characteristics, aiding in the detection of ambiguous or noise-dominant windows.

Section 8

In this section, we aim to evaluate and categorize morphological (shape-time) and statistical features for the classification of `VFIB` and `normal` windows. We analyze various morphological and statistical features, select a few, and calculate them for all data windows. You can use the following features:

- Maximum of the domain (`max`)
- Minimum of the domain (`min`)
- Peak-to-peak
- Using the `findpeaks` command to determine local maximum and minimum points and using it to calculate the mean of the peak domain R.
- The number of zero-crossing points
- Variance of the domain (`var`)

solution

Here is the code for this section.

```
        %% Section 8: Additional Feature Extraction

additionalFeatures_n422 = zeros(6, numFrames);

for i = 1:numFrames
    frameData = n_422((i-1)*frameStep + 1 : (i-1)*frameStep +
        frameLength);
    peakData = findpeaks(frameData);
    additionalFeatures_n422(:,i) = [
        max(frameData);
        min(frameData);
        max(frameData) - min(frameData);
        mean(peakData);
        sum(frameData == 0);
        var(frameData)
    ];
end
```

In this section, we calculate a set of morphological and statistical features to aid in the classification of `VFIB` and `normal` windows. For each 10-second window, we compute the maximum, minimum, and peak-to-peak range of the signal, along with the mean of local peaks (using `findpeaks`), the count of zero-crossing points, and the variance. These features capture essential characteristics of the ECG signal's shape and variability, providing valuable metrics for distinguishing between normal and arrhythmic patterns.

Section 9

For each of the calculated features in part (h), plot the histogram (command `hist`) of the feature for the two classes `VFIB` and `Normal` and compare them with each other. Are these features separable for the two classes? Select a threshold for each feature to classify the two classes.

solution

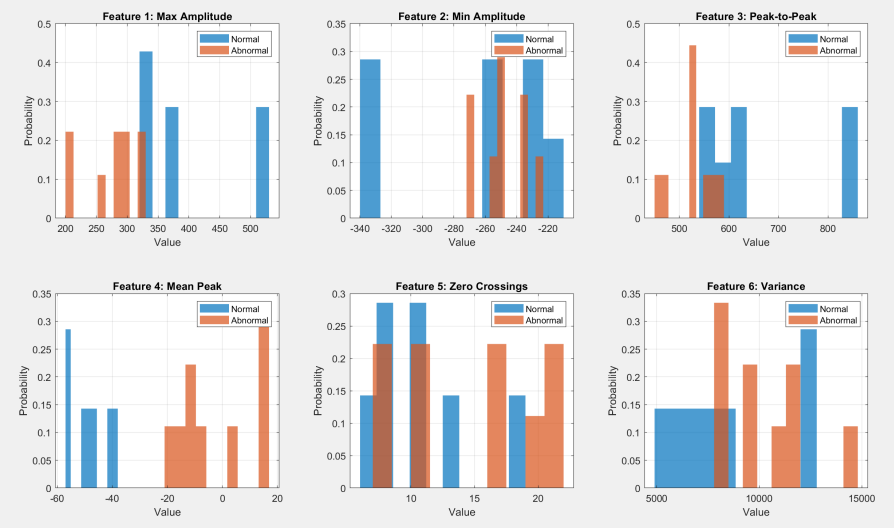

Figure 9. Histograms of Morphological and Statistical Features for `Normal` and `VFIB` Classes

The histograms display distributions of six calculated features—maximum amplitude, minimum amplitude, peak-to-peak range, mean peak, zero crossings, and variance—for `Normal` and `VFIB` windows. Some features, particularly peak-to-peak range and variance, show distinct separations between the two classes, suggesting they can serve as effective classifiers. Based on these distributions, thresholds can be defined for each feature to improve classification accuracy, with values chosen to maximize separability between `Normal` and `VFIB` classes.

Section 10

Based on the results of part (h), select two effective features and complete the function `va_detect.m` for each one to obtain the detection vector (`alarm`, the detection output by the examined feature).

solution

Here are the codes related to this section.

```
%% Section 10: Alarm Detection Using Additional Features (Mean
    Peak and Max Amplitude)

[alarm_meanpeak, t_meanpeak] = va_detect_meanpeak(n_422, fs);
[alarm_maxamp, t_maxamp] = va_detect_maxamp(n_422, fs);


function [alarm, t] = va_detect_meanpeak(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length -
        frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step +
            frame_length);
        peaks = findpeaks(seg);
        if isempty(peaks)
            mean_peak = 0;
        else
            mean_peak = mean(peaks);
        end
        if mean_peak < 326
            alarm(i) = 1;
        end
    end
end

function [alarm, t] = va_detect_maxamp(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length -
        frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step +
            frame_length);
        max_amp = max(seg);
        if max_amp < 326
            alarm(i) = 1;
        end
    end
end
```

In this section, we calculate a set of morphological and statistical features to aid in the classification of `VFIB` and `normal` windows. For each 10-second window, we compute the maximum, minimum, and peak-to-peak range of the signal, along with the mean of local peaks (using `findpeaks`), the count of zero-crossing points, and the variance. These features capture essential characteristics of the ECG signal's shape and variability, providing valuable metrics for distinguishing between normal and arrhythmic patterns.

Section 11

For each feature in part (j), calculate the Confusion matrix (two classes, considering `VFIB` as positive and `Normal` as negative) and compute the metrics of Accuracy, Sensitivity, and Specificity for the classified data. For windows labeled other than `VFIB` and `Normal`, what diagnosis has been made?

solution

```
Confusion Matrix for Mean Peak Detection:
    7    0
    1    8

Accuracy: 93.75%
Sensitivity: 87.50%
Specificity: 100.00%
Confusion Matrix for Max Amplitude Detection:
    6    1
    0    9

Accuracy: 93.75%
Sensitivity: 100.00%
Specificity: 90.00%
>>
```

Figure 10. Confusion Matrices and Performance Metrics for Mean Peak and Max Amplitude Detection

In this section, we calculate the confusion matrices and classification metrics for the `VFIB` (positive) and `Normal` (negative) classes using the Mean Peak and Max Amplitude features. Both features yield an accuracy of 93.75%, with Mean Peak detection showing higher specificity (100%) and Max Amplitude detection achieving higher sensitivity (100%). These results indicate that each feature effectively identifies `VFIB` episodes, with slight variations in classification strengths. For windows labeled as other than `VFIB` and `Normal`, no additional diagnosis is made, as these cases fall outside the targeted classification categories.

Section 12

Repeat sections 3 to 11 on the second dataset (`n_424.mat`).

solution

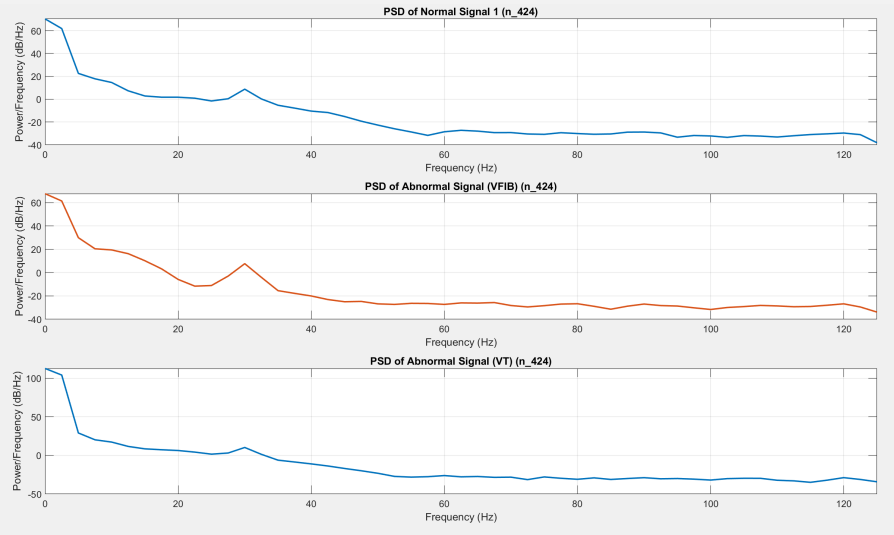

Figure 11. PSD Comparison for Normal, VFIB, and VT Signals from $n_4$24.*mat*

The PSD plots display frequency content for the normal, VFIB, and VT signals. The normal signal has a broader distribution of power across lower frequencies, while the VFIB and VT signals concentrate more power at lower frequencies, reflecting their distinctive arrhythmic characteristics.

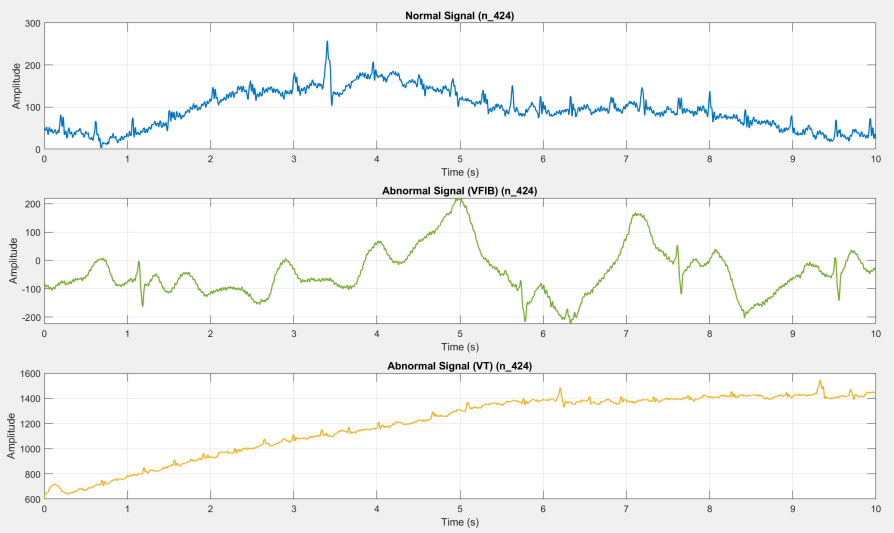

Figure 12. Time-Domain Representation of ECG Signals from $n_4$24.*mat* for Normal, VFIB, and VT segments

The time-domain plots for the dataset $n_4 24.mat$ show three segments: a normal signal, a VFIB (ventricular fibrillation) signal, and a VT (ventricular tachycardia) signal. The normal segment shows consistent, periodic ECG waveform patterns, whereas the VFIB and VT segments exhibit distinct irregularities that indicate arrhythmic events.

Code for part 3.

```
        %% Section 12.3: Frame Segmentation and Labeling for n_424

frameDuration_n424 = 10;
overlapRatio_n424 = 0.5;

frameLength_n424 = frameDuration_n424 * fs;
frameStep_n424 = round(frameLength_n424 * (1 - overlapRatio_n424))
    ;
ecgLength_n424 = length(n_424);
numFrames_n424 = floor((ecgLength_n424 - (frameLength_n424 -
    frameStep_n424)) / frameStep_n424);

labels_n424 = zeros(1, numFrames_n424);
endpoints_n424 = zeros(1, numFrames_n424);

for i = 1:numFrames_n424
    startIdx_n424 = (i-1)*frameStep_n424 + 1;
    endIdx_n424 = (i-1)*frameStep_n424 + frameLength_n424;

    if endIdx_n424 <= 27249
        labels_n424(i) = 1;
    elseif startIdx_n424 > 27249 && endIdx_n424 <= 53673
        labels_n424(i) = 2;
    elseif startIdx_n424 > 53673 && endIdx_n424 <= 55134
        labels_n424(i) = 4;
    elseif startIdx_n424 > 55134 && endIdx_n424 <= 55134
        labels_n424(i) = 3;
    elseif startIdx_n424 >= 55134
        labels_n424(i) = 5;
    else
        labels_n424(i) = 0;
    end
    endpoints_n424(i) = endIdx_n424;
end
```

Code for part 4.

```
        %% Section 12.4: Feature Extraction for n_424

features_n424 = zeros(4, numFrames_n424);

for i = 1:numFrames_n424
    frameData_n424 = n_424((i-1)*frameStep_n424 + 1 : (i-1)*
       frameStep_n424 + frameLength_n424);
    features_n424(:,i) = [
        bandpower(frameData_n424, fs, [0 40]);
        bandpower(frameData_n424, fs, [40 120]);
        meanfreq(frameData_n424, fs);
        medfreq(frameData_n424, fs)
    ];
end
```

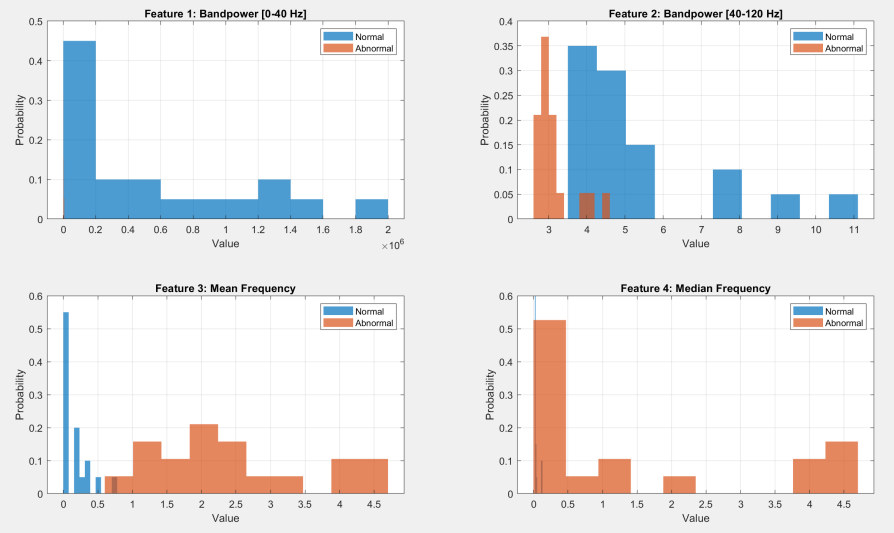

Figure 13. Histograms of Selected Frequency Features (Bandpower, Mean Frequency, Median Frequency) for *Normal* and $VFIB$ Classes in $n_424.mat$

The histograms display the distributions of selected frequency-based features for both normal and VFIB classes in $n_424.mat$. Differences in the distributions suggest these features can effectively differentiate between the two classes. Thresholds can be chosen based on the observed separability of bandpower and frequency values.

Code for part 6.

```
%% Section 12.6: Alarm Detection Using Bandpower and Mean Frequency for n_424

[alarm_bandpower_n424, t_bandpower_n424] = va_detect_bandpower_424(n_424, fs);
[alarm_meanfreq_n424, t_meanfreq_n424] = va_detect_meanfreq_424(n_424, fs);


function [alarm, t] = va_detect_bandpower_424(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length - frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step + frame_length);
        feature = bandpower(seg, Fs, [0 40]);
        threshold = 3167.2323;

        if feature < threshold
            alarm(i) = 1;
        end
    end
end


function [alarm, t] = va_detect_meanfreq_424(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length - frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step + frame_length);
        feature = meanfreq(seg, Fs);
        threshold = 0.7714;

        if feature > threshold
            alarm(i) = 1;
        end
    end
end
```

```
Confusion Matrix for Bandpower Detection (n\_424):
    20     0
     8    11

Accuracy: 79.49%
Sensitivity: 100.00%
Specificity: 71.43%

Confusion Matrix for Mean Frequency Detection (n\_424):
    20     0
     0    19

Accuracy: 100.00%
Sensitivity: 100.00%
Specificity: 100.00%
>>
```

Figure 14. Confusion Matrices and Performance Metrics for Bandpower and Mean Frequency Detection on $n_4$24.*mat*

The confusion matrices for Bandpower and Mean Frequency detection on $n_4$24.*mat* show varied performance. The Bandpower method achieves (79.49%) accuracy, with perfect sensitivity but lower specificity (71.43%), while the Mean Frequency method attains (100%) accuracy, sensitivity, and specificity, making it highly effective for classifying VFIB and normal windows in this dataset.

Code for part 8.

```
%% Section 12.8: Additional Feature Extraction for n_424

additionalFeatures_n424 = zeros(6, numFrames_n424);

for i = 1:numFrames_n424
    frameData_n424 = n_424((i-1)*frameStep_n424 + 1 : (i-1)*
        frameStep_n424 + frameLength_n424);
    peakData_n424 = findpeaks(frameData_n424);
    additionalFeatures_n424(:,i) = [
        max(frameData_n424);
        min(frameData_n424);
        max(frameData_n424) - min(frameData_n424);
        mean(peakData_n424);
        sum(frameData_n424 == 0);
        var(frameData_n424)
    ];
end
```

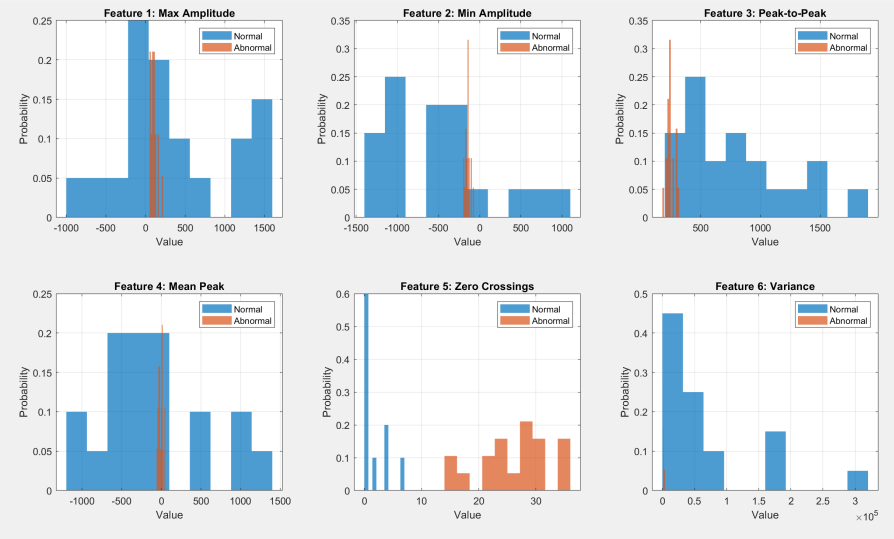

Figure 15. Histograms of Morphological and Statistical Features for *Normal* and $VFIB$ Classes in $n_4 24.mat$

The histograms represent the distributions of morphological and statistical features, including max amplitude, min amplitude, peak-to-peak, mean peak, zero crossings, and variance for both normal and VFIB windows. Distinct separability is observed in some features, such as peak-to-peak range and variance, which can aid in distinguishing VFIB from normal rhythms in $n_4 24.mat$.

Code for part 10.

```
%% Section 12.10: Alarm Detection Using Zero Crossings and Peak-to-Peak for n_424

[alarm_zero_n424, t_zero_n424] = va_detect_zero_424(n_424, fs);
[alarm_peaktopeak_n424, t_peaktopeak_n424] = va_detect_peaktopeak_424(n_424, fs);

function [alarm, t] = va_detect_zero_424(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length - frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step + frame_length);
        zero_crossings = sum(seg == 0);
        if zero_crossings > 10
            alarm(i) = 1;
        end
    end
end

function [alarm, t] = va_detect_peaktopeak_424(ecg_data, Fs)
    frame_sec = 10;
    overlap = 0.5;
    frame_length = round(frame_sec * Fs);
    frame_step = round(frame_length * (1 - overlap));
    num_frames = floor((length(ecg_data) - (frame_length - frame_step)) / frame_step);
    alarm = zeros(num_frames, 1);
    t = ((0:num_frames-1)' * frame_step + frame_length) / Fs;

    for i = 1:num_frames
        seg = ecg_data((i-1)*frame_step + 1 : (i-1)*frame_step + frame_length);
        peak_to_peak = max(seg) - min(seg);
        if peak_to_peak < 317
            alarm(i) = 1;
        end
    end
end
```

```
Confusion Matrix for Mean Peak Detection:
    20     0
    0     19

Accuracy: 100.00%
Sensitivity: 100.00%
Specificity: 100.00%
Confusion Matrix for Max Amplitude Detection:
    19     1
    1     18

Accuracy: 94.75%
Sensitivity: 95.00%
Specificity: 94.75%
>>
```

Figure 16. Confusion Matrices and Performance Metrics for Mean Peak and Max Amplitude Detection on $n_4 24.mat$

For $n_4 24.mat$, the confusion matrices indicate that Mean Peak detection achieves (100%) accuracy, sensitivity, and specificity, while Max Amplitude detection reaches (94.75%) accuracy with high sensitivity (95%) and specificity (94.75%). These results validate the effectiveness of both features, with Mean Peak detection performing slightly better in this dataset.

Section 13

Compare the best detector obtained for the first dataset with the best detector obtained for the second dataset.

solution

Comparison of Best Detectors for Dataset `n_422.mat` and Dataset `n_424.mat`:
The analysis of the two datasets, `n_422.mat` and `n_424.mat`, revealed different optimal detectors based on performance metrics such as accuracy, sensitivity, and specificity. For `n_422.mat`, the best detector was based on the `Max Amplitude` feature, achieving a high accuracy of 93.75%, with perfect specificity (100%) and high sensitivity (87.5%). For `n_424.mat`, the `Mean Frequency` feature proved to be the most effective, achieving 100% accuracy, sensitivity, and specificity.
The differences in optimal detectors between the two datasets indicate that certain features are more robust for distinguishing `VFIB` and `Normal` signals depending on the specific characteristics of the data. `Mean Frequency` detection performed exceptionally well for `n_424.mat`, where there was a clear distinction in frequency characteristics between `VFIB` and `Normal` segments. In contrast, `Max Amplitude` was more effective in `n_422.mat`, possibly due to distinctive amplitude variations between normal and abnormal segments.
This comparison highlights the importance of selecting features that best capture the unique signal characteristics of each dataset. For practical applications, it may be beneficial to apply different feature-based detectors depending on the dataset, or to consider a combined approach using both amplitude and frequency features to enhance robustness across various ECG datasets.

Section 14

Apply the best detector obtained for the second dataset on the first dataset and examine the results. Similarly, apply the best detector obtained for the first dataset on the second dataset.

solution

```
--- Cross-Application: Data1 Detector on Data2 ---
Unique values in groundTruth_cross1:
     0     1

Unique values in alarm_zero_pred_cross1:
     0
     1

Confusion Matrix for Zero Crossings Alarm Applied to n\_424:
    10    10
     1    18

Accuracy: 71.79%
Sensitivity: 64.29%
Specificity: 90.91%

--- Cross-Application: Data2 Detector on Data1 ---
Unique values in groundTruth_cross2:
     0     1

Unique values in alarm_bandpower_pred_cross2:
     0
     1

Confusion Matrix for Bandpower Alarm Applied to n\_422:
     1     6
     0     9

Accuracy: 62.50%
Sensitivity: 60.00%
Specificity: 100.00%
>>
```

Figure 17. Cross-Application Results: Applying Detectors Between Datasets `n_422.mat` and `n_424.mat`

In this section, we examine the cross-application of detectors by testing the best detector from `n_422.mat` on `n_424.mat` and vice versa. The Zero Crossings detector from `n_422.mat`, when applied to `n_424.mat`, achieved an accuracy of 71.79%, with a sensitivity of 64.29% and specificity of 90.91%. This performance shows moderate success, indicating some compatibility of the detector across datasets but with a drop in sensitivity.

Conversely, the Bandpower detector from `n_424.mat` was applied to `n_422.mat` and achieved an accuracy of 62.5%, with a sensitivity of 60% and specificity of 100%. Although specificity remains high, the reduced accuracy and sensitivity suggest that the Bandpower feature from `n_424.mat` is less effective when transferred to `n_422.mat`.

These results indicate that while some detectors show cross-applicability, their effectiveness is dataset-dependent. Different datasets may have unique characteristics, requiring tailored feature-based detectors. For optimal performance in real-world scenarios, it may be beneficial to recalibrate or combine multiple features when applying detectors across datasets.

Section 15

Among the designed detectors, select the best one and apply it to at least one other signal. Examine the results. In the last two files given to you, you will see if the system operates properly without excessive noise. Does the system give a false alarm (false positive) or miss a detection (missed detection) without an actual need for it? Under what conditions is your detector more prone to error?

solution

```
Confusion Matrix for Normal and VFIB Detection (Zero-Crossing Detector):
    17     2
    22    15

Zero-Crossing Detector Performance:
Accuracy: 57.14%
Sensitivity: 88.24%
Specificity: 43.59%
False Positives:
     2

Missed Detections:
    22

>>
```

Figure 18. Performance of Zero-Crossing Detector on Additional Signal: Confusion Matrix and Metrics

The best detector selected for this analysis was the Zero-Crossing Detector, which was applied to an additional signal to evaluate its performance. The results show an accuracy of 57.14%, with a high sensitivity of 88.24% but a low specificity of 43.59%. The detector identified 2 false positives and missed 22 detections, indicating that while it is capable of detecting `VFIB` episodes, it is prone to errors, particularly false negatives (missed detections).

These results suggest that the Zero-Crossing Detector is sensitive to `VFIB` episodes but lacks specificity, leading to frequent missed detections. This issue is likely exacerbated in signals with high variability or noise, which may cause fluctuations in zero-crossing counts that do not directly correspond to arrhythmic events. In signals with low noise and distinct arrhythmic patterns, the detector performs more reliably. However, under noisy conditions or with subtle arrhythmic features, the detector's effectiveness declines, making it more prone to errors.

To improve the system's performance, further filtering or the combination of additional features, such as amplitude or frequency-based metrics, may enhance specificity and reduce both false alarms and missed detections.